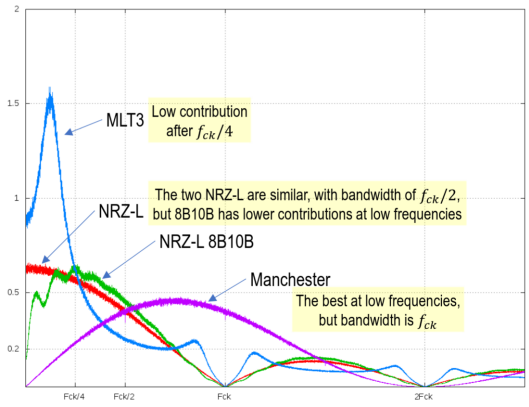
1 Lesson 30-10-26

1.1 Spectrum of serial protocols

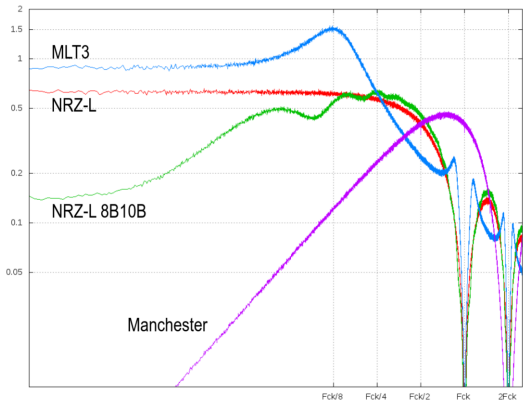
Let's look at some *waveform* and *spectrum* examples for serial communications protocols.

DC and low frequency components may be hard to transmit on most channels, especially if there are galvanic or optical transformers in the transmission chain. Some encodings guarantee the absence of DC and low frequency components, such as *Manchester* and *8B10B*, which guarantees an equal number of 0s and 1s in the encoding in the long run (we might have said something about this in the past lesson).

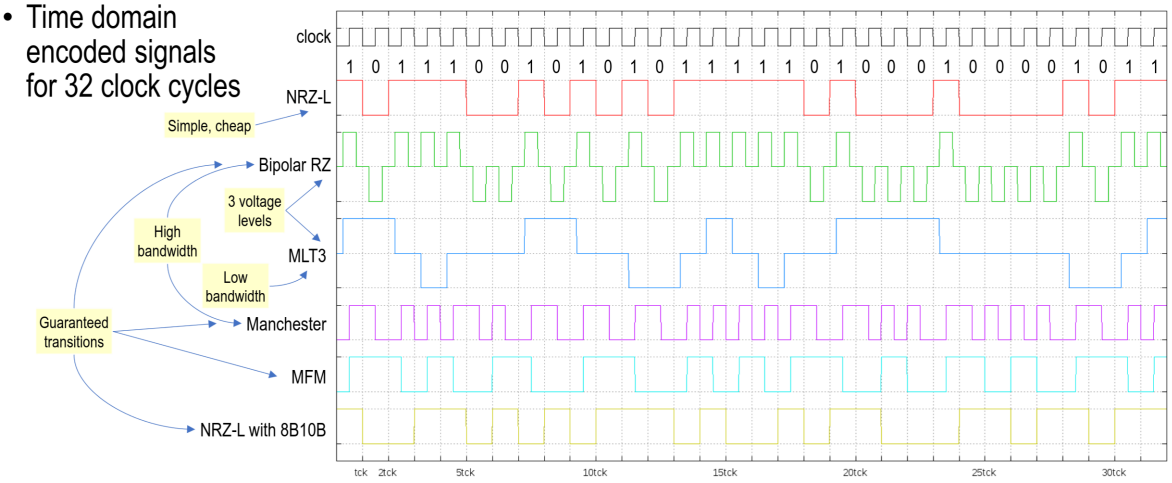
The sepctra of serial data line waveforms shows a superimposed sinc trend (Fourier transform of a box function), which is zero at f_{ck} and multiples. Regarding the **bandwidth**, this means there are still major contributions to the signal above the bandwidth if taken at f_{ck}.



Linear scales



Logarithmic scales (similar to Bode)



1.2 SPI and I2C

These are standards used to connect integrated circuits, on the same board, or on separate boards (over short distances), organized as *networks* of nodes.

Master and **slave** devices are identified: they have different roles, but they can both be receiver and/or transmitter.

They are of course *serial*, but usually offer a dedicated clock line. The separate wire for clock transmission is a skew problem, as we cannot achieve very high performance (from hundreds of Kbit/s to a few Mbit/s), and the length of the interconnection is limited. The fact that multiple nodes can drive the single serial line, on the other hand, save costs and allows to have less wires.

1.2.1 SPI

SPI (*Serial Peripheral Interconnect*), also known as the 3 wires protocol, is a serial transmission protocol supporting:

- Single master, multiple slaves, where the number of slaves is in principle unlimited (in practice usually limited to 4 or 5);
- It allows for **full duplex**, with one wire for data from master to slaves (**MOSI**, *Master Out Slave In*), and one wire for data from slaves to master (**MISO**, *Master In Slave Out*).

  Master and slaves are always both RX and TX.

So, we need at least 3 wires, but often we find 4 or more:

- **SPICLK**, the clock, always generated by the master;
- **MOSI**, Master Out Slave In, the output data line of the master, and input data line of slaves;
- **MISO**, Master In Slave Out, the input data line of the master, and output data line of slaves. It can have multiple drivers, so it needs to be tristated, and there are risk of dangerous conflicts;
- **SPISS**, not strictly necessary, it's the *SPI Slave Select*. It is connected to the *Chip Select* input of slaves, and driven by the master.

  There might be multiple SPISS when connecting to several slaves, or it might be absent when the master is connected to a single slave. The Chip Select of a slave is always active (i.e. connected to GND), and this overrides it.

  If it's the master that orchestrates slave communications, then there is no (or at least less) risk of two slaves deciding to talk at the same time. This terminology is gross.

On data transmission, a predetermined number n of bits is transmitted. All blocks in the SPI network have internal shift registers of size n to allow this, and one bit per clock cycle is transmitted. *NRZ encoding* is used, as its simple and cheap. No transition is guaranteed, but this is fine as the separate clock provides the required synchronization. SPICLK is idle when there is no transmission, at a constant level defined by the *clock polarity* **CPOL**.

Let's look at a transmission

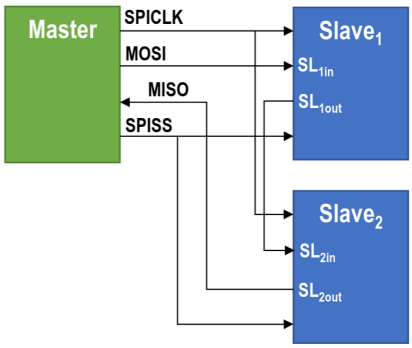
1. The Slave Select of the target slave is activated during the entire transmission, and on all other slaves the SS is not active. These keep their output (on MISO) in high impedance, and ignore what happens on MOSI and SPICLK.
2. The master generates n clock cycles: the first clock transition denotes the beginning of a transmission. Data is transmitted MSB first, changes on a clock edge and is sampled on the opposite clock edge, according to the *clock phase* **CPHA**.
3. The clock returns idle at the end of the transmission;
4. Slave Select is deasserted to complete the transmission.

Notice that CPOL and CPHA define three parameters:

- Idle clock level
- Active edge for data output
- Active edge for data sampling

CPOL	CPHA	Idle CLK	Data output	Sampling
0	0	0	Falling	Rising
0	1	0	Rising	Falling
1	0	1	Rising	Falling
1	1	1	Falling	Rising

SPI supports **daisy chaining**. SPISS not strictly needed, and there is no shared MISO: all slaves get the clock, but only one gets MISO. The other slaves talk one to another, and only the slave which gets the MISO talks directly to the master. Again, gross.



Note that every transmission involves all slaves in this configuration, as values have to propagate through slaves to the slave connected to MISO, and thus the master.

To summarize SPI:

- Arbitration is not needed, as there is only a single master, and slaves cannot initiate a transmission. All arbitration is done by the master;
- Addressing: the target slave is chosen using Chip Select signals,o ne separate signal for each slave, expensive and not easy to change after fabrication. In daisy chain connection all slaves are always involved;
- Bit synchronization: clock always generated by the master, configurable polarity and active edges. A slow slave cannot stop the master.